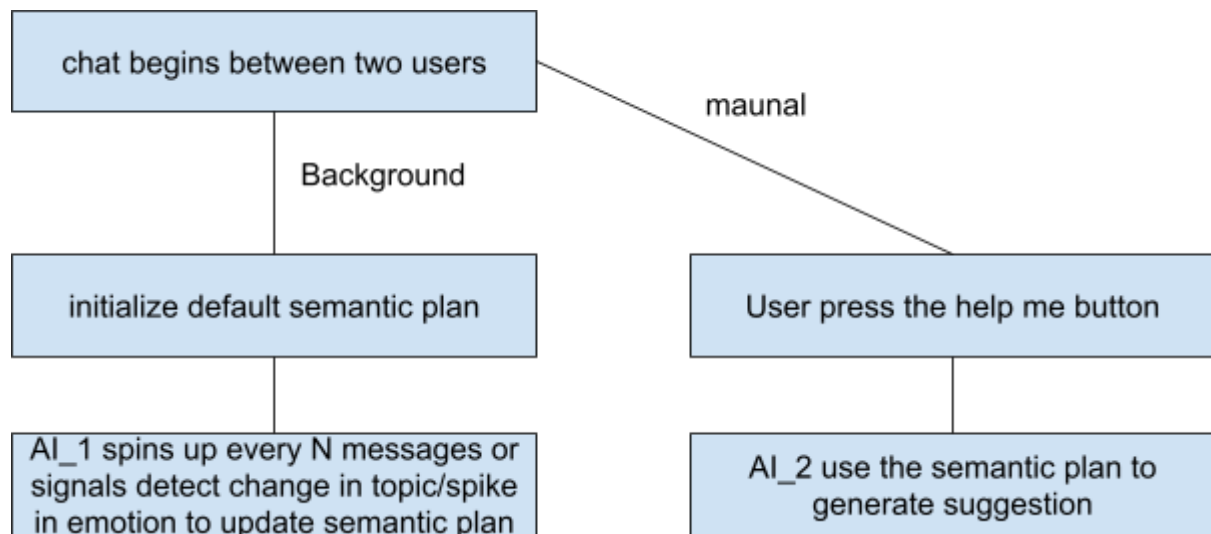


System Objective & Core Mechanics

Goal: To support users in live chat environments with personalized, context-aware conversational suggestions while actively preventing AI dependency.

The system employs a progressive degradation strategy. Suggestions avoid explicit dialogue scripting. As the chat health improves, the AI visually degrades and structurally shifts its output from concrete advice to vague, meta-cognitive prompts. If the chat deteriorates, the system regresses to higher visibility and concrete support.

steps:



Decoupled Pipeline Architecture

To achieve sub-second real-time latency while executing complex psychological analysis, the suggestion pipeline is strictly decoupled into two distinct stateless API processes. All user profiling and conversational context are stored in an Appwrite NoSQL database.

Agent 1:

Operates silently to maintain the conversational state without impacting UI latency.

- **Trigger:** Executes periodically based on message count thresholds (e.g., every 15 messages) or heuristic spikes (eg.increase in response time or major shift in mood/ topic)
- **Execution:** A stateless backend function fetches the user profiles, the current chat log, and the active **Semantic Plan** JSON document from Appwrite. It passes these to a heavy LLM via a direct API call to evaluate chat health, record chat context and

generate plans of action, upon initial creation of the chat, a default semantic plan template will be created to be used as the opening.

- **Output:** Updates the **Semantic Plan** in Appwrite
- **Garbage Collection:** Appwrite's native **\$updatedAt** metadata governs context. If a chat is idle for > 12 hours, the backend forces a context reset. A scheduled CRON job permanently deletes Semantic Plans older than 30 days.

Knowledge Graph implementation:

in addition to the macro summary in the semantic plan, we will also keep an overall knowledge graph of the whole chat:

this will be done by using llamaindex and mongoDB, first when agent 1 activated the semantic plan update, the system prompt will be separated into two tasks, one is the original update the semantic plan, the second will be to use that section of text to update/construct the knowledge graph, first through llamaindex, it bundles the text with system prompt that states the allowed list of relationships, then we send the payload to the ollama model, then the model will reply with: an updated semantic plan that will be upserted to the DB, and a JSON array of Graph Triples.

after getting the triples, we first check if all the relationships are in the prompt list, if a relationship is not in the list, it drops it, after that, It loops through the JSON triples. splits it into its entities: the Subject and the Object, It sends two separate **Upsert** commands to the MongoDB **Nodes** collection:

1. *"Hey Mongo, look for **user_brian**. If he exists, update his **last_mentioned** clock to right now. If he doesn't exist, create a new profile for him."*
2. *"Hey Mongo, look for **fade_out_system**. If it exists, update its clock. If it doesn't, create it."*

Now the backend draws the line. It takes the full triple and generates a unique ID for that specific connection

It sends one final **Upsert** command to the MongoDB **Edges** collection:

- *"Hey Mongo, look for an edge with this exact ID connecting these two specific nodes. If it does NOT exist, draw the line and set its **weight** to 1. If it ALREADY exists, just update its clock and increment its **weight** by +1."*

The loop finishes. The backend drops the variables from its active memory to free up RAM, closes the database connection, and Agent 1 goes dormant until the chat room hits the next 5-message threshold.

Agent 2

Operates exclusively when the user manually triggers the "Help me reply" UI.

- **Trigger:** Manual user invocation.

- **Execution:** Acts strictly as a data-formatting engine. The backend injects the dynamic boundaries from Agent 1 and a 3-5 message Sliding Window into a lightweight LLM API call.
- **Output:** Returns a strictly formatted JSON payload containing the UI nudge in under 1.5 seconds.

Based on how the chat is going, agent 2 will take on a number of different roles, which will affect how the suggestion is generated:

starting out	smooth	struggling	neutral
friend of a friend(letting you know about the other person and maybe a few conversation starter, but then dips)	mentor(letting user try themselves while giving more vague hints or even just observing)	adviser(more personalized and helpful advice, should still avoid telling user exactly what to say no matter what)	Facilitator. Provide one nudge to sustain the current light topic, and one optional, low-pressure question to pivot slightly deeper.

to transition between these roles, see Deterministic state management matrix below

what the frontend sends to agent 2 in the backend:

```
{
"chat_id": "chat_5512_9941",
"requesting_user_id": "u_5512",
"opposing_user_id": "u_9941",
"current_draft": "", //what is currently in the input bar
"t_invoke_ms": 2800 // Frontend calculation: Time since last message was rendered
}
```

the agent 2 alongside this will also read the semantic plan, as well as a sliding window context of n-1 most recent messages, n being the number of messages to trigger agent 1

using this it returns a json output of the suggestion

querying the knowledge graph:

with the addition of the knowledge graph, the agent 2 can now use it to find related topics, to do this:

When the user clicks the "Suggest" button, the backend scans the sliding window of the recent chat history to extract a ranked list of conversational entities, prioritizing topics by how frequently and recently they were mentioned. Using these active anchors, the backend queries the MongoDB Knowledge Graph to retrieve a highly targeted sub-graph of historical facts, shared experiences, and established preferences connected to the user and their partner. The backend then constructs a master prompt that injects this retrieved historical lore alongside the Semantic Plan and the immediate context of the sliding window itself. Finally, the LLM reads this combined context, synthesizing the macro-strategy with the specific historical details to generate personalized, contextually accurate dialogue suggestions that seamlessly match the current flow of the conversation.

Here is an interactive visualization demonstrating exactly why your intuition to use a sliding window prevents Agent 2 from getting derailed by brief changes in conversation.

3-Layer Risk & Boundary Enforcement (Updated Architecture)

To guarantee safety and prevent maladaptive cognitive offloading, the system utilizes a cascading risk mitigation strategy with strict Gating/Guardrails at every level.

Layer 1: The Pre-Generation Hard Gate (Toxicity & Spam)

- **Mechanism:** User inputs are scanned by a lightweight risk detection algorithm before the AI is invoked.
- **Action:** If a message triggers the risk threshold (e.g., harassment, spam), the system immediately intercepts the action and reassigns the user's `current_role` to **RISKY** (The Quarantine State).
- **The Quarantine Gating/Guardrails:**
 - **Generation Locked:** Agent 2's generative capabilities are completely disabled. The system returns a hardcoded warning: *"Message blocked: Violates community guidelines. Assist disabled."* The AI will not generate apologies or pivot suggestions.
 - **The Strike System:**
 - **Strike 1:** User is locked in the RISKY state. To exit, they must manually type and send **5** consecutive clean messages.
 - **Strike 2:** Quarantine locked. The threshold doubles to **10** consecutive clean messages.
 - **Strike 3:** Terminal node. The chat room is permanently destroyed, and the user's account is flagged for review/banning.

Layer 2: API-Level Structural Guardrailing (Anti-Plagiarism & Role Bounds)

- **Mechanism:** The backend injects absolute laws into Agent 2's System Prompt and forces the LLM to output strictly formatted JSON. The schema physically restricts what the AI is capable of generating based on the current matrix state.
- **The Global Guardrail (All Roles):** "You are strictly forbidden from outputting exact dialogue, scripts, or using quotation marks. You must only output meta-cognitive instructions or structural frameworks."
- **The Role-Specific Guardrails Matrix:**
 - 1. FRIEND (Cold Start / < 5 Messages)**
 - **Prompt Injection:** "Role: FRIEND. Generate a conceptual icebreaker. You must incorporate at least one data point from the 'user_2_context' JSON (e.g., studying Information Management, location, hobbies). Do not offer emotional advice."
 - **Required Schema:**
 - JSON

```
{
  "nudge_type": "icebreaker",
  "target_fact": "[Fact from profile]",
  "nudge_text": "Ask them about their..."
}
```

-
-
- **2. FACILITATOR (Healthy Health / Low Dependency)**
 - **Prompt Injection:** "Role: FACILITATOR. The users are succeeding. Provide a minimalist thematic nudge. Your **nudge_text** MUST be under 8 words. Focus only on the immediate topic."
 - **Required Schema:**
 - JSON

```
{
  "nudge_type": "thematic_push",
  "nudge_text": "Pivot to [Topic] or Ask about [Detail]"
}
```

-
-
- **3. ADVISER (Failing Health)**
 - **Prompt Injection:** "Role: ADVISER. The conversation is stalling. Provide a structural, two-step framework for their next message. Tell them the 'What' and the 'Why', not the exact words."
 - **Required Schema:**

- JSON

```
{
  "nudge_type": "structural_lifeline",
  "step_1_acknowledge": "Validate their feeling about...",
  "step_2_pivot": "Transition the topic toward..."
}
```

-
-

- **4. MENTOR (High Dependency Penalty)**

- **Prompt Injection:** "Role: MENTOR. The user is offloading their cognitive effort. YOU MUST NOT SUGGEST A REPLY. Output a single Socratic question directed at the user to force them to process the match's last message."
- **Required Schema:**
- JSON

```
{
  "nudge_type": "socratic_block",
  "internal_question": "What is your actual opinion on [Subject]?",
  "ui_instruction": "Answer this for yourself before replying to them."
}
```

5. The RISKY (Quarantine)

- **The Goal:** Stop malicious behavior.
- **The Constraint:** As we established, Agent 2 is completely locked out here.
- **Layer 2 Action:** The backend intercepts the `current_role == "RISKY"` flag. It does not even make the API call to the LLM. It immediately returns the hardcoded hardgate error payload to the frontend.

Layer 3: Output Parsing & The Sieve (Differentiated Error Handling)

- **Mechanism:** The backend validates Agent 2's JSON response before sending it to the client. It filters the output through a strict Sieve to differentiate between a harmless API glitch and a user violating the rules (e.g., Prompt Injection).
- **The Sieve Checks:**

- Is it valid JSON matching the exact Layer 2 schema?
- Does the text contain a substring perfectly matching previous messages (Plagiarism)?
- Does it trip the safety/toxicity filters?
- **Action (The API Payload Split):**
 - **Path A: The Softgate (System Fault)**
 - *Trigger:* LLM API timeout or a minor JSON formatting error (missing comma).
 - *Backend Output:*
 - JSON

```
{
  "status": "softgate_retry",
  "error_type": "parsing_timeout",
  "ui_message": "Network hiccup. Tap to try again."
}
```

-
-
- *Frontend Action:* UI allows the user to click the button again to re-trigger the generation.
- **Path B: The Hardgate (User Fault / Boundary Violation)**
 - *Trigger:* The LLM outputs exact dialogue, breaks the role schema, or the user attempts a prompt injection.
 - *Backend Output:*
 - JSON

```
{
  "status": "hardgate_locked",
  "error_type": "boundary_violation",
  "ui_message": "Action disabled: AI cannot generate exact replies. You must type this message manually."
}
```

- *Frontend Action:* The AI UI physically disables the button and removes the event listener. The button cannot be clicked again for this conversational turn. The user is forced to carry the cognitive load and type the message manually.

Deterministic State Management & Signal Processing Matrix

Agent 2 does not possess the autonomy to select its role or conversational strategy. The backend administrator strictly dictates the role by calculating raw behavioral signals to update the `current_role` string in the Appwrite database.

To ensure low-latency performance and minimize LLM overhead, the system relies on quantitative heuristics acting as a "smoke alarm."

The Signal

The backend tracks two primary arrays of metadata without needing to run heavy NLP sentiment analysis on every message:

1. The Health Signals (\$H\$): Tracks organic conversational stability.

- Response Latency: The time delta between messages. Acts as the primary proxy for cognitive engagement.

- how to calculate: latency is the difference in timestamp between two chats in sequential order(msg_id), if there is only one message set default to null, for overall health we use a EMA with alpha = 0.15

$$H_{lat_ema} = (L_{current} \times \alpha) + (Previous_H_{lat_ema} \times (1 - \alpha))$$

\$L_{current}\$: The time elapsed (in seconds) between the previous message's `created_at` timestamp and the new message's `created_at` timestamp.(important to note all time unit should be turned in to seconds to prevent crashing)

\$Previous_H_{lat_ema}\$: The stored value pulled from the Appwrite `semantic_plan`. (Default to the very first `$L_{current}$` value for a brand new chat).

\$\alpha\$: The smoothing factor. Because this is tracking *gradual* macro health, slow EMA that doesn't overreact to a single message.

- Message Length Parity: The character count ratio between users. Detects conversational narcissism and ensures mutual social effort.
 - how to calculate: the difference in char_length between two messages in sequential order(msg_id), if there is only one message set default to null, for overall chat health use an EMA with alpha = 0.15
 - **Edge case:** introduce a minimum character floor before parity calculations are weighted, or combine parity with an absolute volume metric (e.g., total characters exchanged in the sliding window).

introduce a minimum character threshold (θ), representing the lowest acceptable total character count for a healthy two-message exchange (e.g., 30 characters). You then multiply your raw parity score by a volume multiplier.

Step 1: Calculate Raw Parity

$$P_{\text{raw}} = \frac{\min(\text{Chars}_A, \text{Chars}_B)}{\max(\text{Chars}_A, \text{Chars}_B)}$$

Step 2: Calculate the Volume Multiplier

$$V_{\text{mult}} = \min\left(1.0, \frac{\text{Chars}_A + \text{Chars}_B}{\theta}\right)$$

(If the total characters meet or exceed your threshold θ , the multiplier caps at 1.0, leaving the raw parity unaffected. If it's below the threshold, it creates a fraction that drags the score down).

Step 3: The Adjusted Equation

$$P_{\text{adjusted}} = P_{\text{raw}} \times V_{\text{mult}}$$

Using the "ok" / "ya" example with a threshold (θ) of 30:

- $P_{\text{raw}} = 1.0$
 - $V_{\text{mult}} = 4 / 30 = 0.133$
 - **Final Parity Score = 0.133**
- - $H_{\text{par_ema}} = (P_{\text{current}} \times \alpha) + (\text{Previous_}H_{\text{par_ema}} \times (1 - \alpha))$
 - **$\text{Previous_}H_{\text{par_ema}}$** : The stored value from Appwrite. (Default to 1.0 for a new chat, assuming perfect parity until proven otherwise).
 - **P_{current}** : The parity of the current exchange. Must always be between 0.0 and 1.0:
 - $P_{\text{current}} = \frac{\min(\text{Chars}_{\text{current_msg}}, \text{Chars}_{\text{previous_msg}})}{\max(\text{Chars}_{\text{current_msg}}, \text{Chars}_{\text{previous_msg}})}$

If $\max = 0$, fallback to 0.0 to avoid a divide-by-zero error

In text-based Computer-Mediated Communication (CMC), the absence of physical non-verbal cues forces time to act as the primary proxy for social engagement. Consequently, the system's backend calculates chat health. In real-time environments, delays are actively attributed to negative relational signals such as rejection or disinterest ([Kalman et al., 2013](#)), and these chronemic lapses directly cause interaction failure ([Lew et al., 2018](#)).

Message Length Parity prevents Conversational Narcissism ([Vangelisti, Knapp, & Daly 1990](#)) a structural communication failure where one participant monopolizes the interaction, forcing the other into a passive audience role. In digital environments, reciprocal text volume is the primary mathematical indicator of Interactive Alignment ([Pickering & Garrod, 2004](#)). Without tracking this character count ratio, the system would falsely classify a chat as

"healthy" purely based on fast reply speeds, ignoring the fact that one user is carrying the entire cognitive load of the conversation.

2. The Dependency Signals (\$R\$): Tracks the user's reliance on the AI.

- AI Call Ratio: Calculates what percentage of the last messages were drafted using the AI.
 - It draws the mathematical boundary between *communication augmentation* (healthy) and *communication replacement* (toxic). If a user uses the AI 1 out of 10 times to get out of a rut, it is a tool. If they use it 8 out of 10 times, they are no longer participating in the conversation; they are a bystander to their own relationship.
 - This tracks the risks of Artificial Intelligence-Mediated Communication (AI-MC), a framework defined by [Hancock, Naaman, and Levy \(2020\)](#). Their research warns that hyper-reliance on AI generation fundamentally alters human interpersonal dynamics. A high AI Call Ratio indicates severe Algorithmic Dependence. When this happens, the authenticity and relational development of the chat collapses, because the connection is technically being formed between the receiving user and your AI, not the two humans.
 - how to calculate:
 - uses **Exponential Moving Average (EMA)**. It naturally decays older uses while heavily weighting recent behavior.

The EMA formula relies on a binary input for the user's current action, which we will call U_{action} .

- If the user sends a message generated by the AI:
 $U_{\text{action}} = 1.0$
- If the user sends an organic message typed themselves:
 $U_{\text{action}} = 0.0$

The Formula:

$$S_{\text{rat_ema}} = (U_{\text{action}} \times \alpha) + (\text{Previous_EMA} \times (1 - \alpha))$$

where the smoothing vector $\alpha = 0.3$ (places 30% of the weight on the very last message, and 70% on the historical context)

default for previous ema = 0.0

- Zero-Keystroke Invocation: A conditional penalty that cross-references the user's semantic input with their wait time (invocation latency) before clicking the AI button.

- It measures the complete absence of initial effort. If a user types "I like movies" and asks the AI to make it sound better, they provide the semantic seed. That is *cognitive scaffolding*. If they open a blank text box and just hit "Generate," they provide absolutely nothing. So we measure this by giving out penalty for not trying it themselves first by:

Input is empty/gibberish AND $ST_{\{invoke\}} < 5$ seconds., why 5 seconds:

The Reading Speed Baseline: According to [Rayner \(1998\)](#) on the psychology of reading, the average adult reads at approximately 238 to 260 words per minute (roughly 4 words per second).

The Motor Action (Fitts's Law, Card, S. K., Moran, T. P., & Newell, A. (1983). *The Psychology of Human-Computer Interaction*. Lawrence Erlbaum Associates.): It takes approximately 1 second(simplification) for a user to cognitively decide to press a button and physically move their thumb to hit it.

If a match sends a standard 10 word message, it takes a human roughly 2.5 seconds just to physically scan the words, plus 1 second of motor action to hit the AI button. That totals roughly 3.5 seconds. We add a bit of buffer for a total of 5 seconds

Therefore, if $ST_{\{invoke\}} < 5$ seconds, it is *physically impossible* that the user read the message, processed its semantic meaning, attempted to formulate an independent response, failed, and then asked for help. A sub-5-second click is a pure, thoughtless reflex. It proves they are bypassing the cognitive process entirely.

The user did not even try to read the message and think of a response. This is true maladaptive cognitive offloading.

- This perfectly aligns with Cognitive Offloading theory ([Risko & Gilbert, 2016](#)). Risko and Gilbert define offloading as using external tools to reduce the cognitive demands of a task. While *strategic* offloading is useful, a Zero-Keystroke Invocation represents **maladaptive cognitive offloading**. It correlates with a total collapse of intrinsic social motivation. The user is attempting to bypass the mental effort of social interaction entirely.
- how to calculate: in the payload for agent 2, it can use the "current_draft" and "t_invoke_ms" to run checks, if it turns out to be true it increment the "s_inv_penalties" signal by 1
 - backend calculation:
 - **Time Check:** Is $ST_{\{invoke\}} < 5000$ ms?
 - **Effort Check (Empty/Gibberish):** * Strip whitespace:
`cleaned_draft = current_draft.trim()`
 - Length check: If the length is less than 3 characters (e.g., "a", "k", "yo"), it counts as zero semantic scaffolding.
 - Keyboard mashing regex: Check for high repetition of the same character or sequence of chars (e.g., "asdfgh" or "jjjjj" or "asdasd").

the signals will be updated every time agent 1 is triggered in the back ground, but decay will also be calculated when agent 2 is activated

C. The State Transition Matrix (Deterministic Routing)

The backend evaluates the four raw signals—Response Latency (S_{lat}), Message Parity (S_{par}), AI Call Ratio (S_{rat}), and the Latency-Conditioned Invocation Flag (S_{inv})—as independent tripwires. The session is pushed through a hardcoded *if/else* hierarchy to assign the current AI role:

Case 1: FRIEND (The Icebreaker)

- **Condition:** $\text{total_messages} < 5$ (parameter, can be changed)
- **Action:** Bypasses all signal evaluations. The cold-start problem poses the highest risk of abandonment for strangers. The system forces high-visibility UI and prioritizes profile matching to establish initial common ground before enough data exists to calculate behavioral baselines.

Case 2: ADVISER (User need help)

- **Condition:** $(H_{\text{lat_ema}} > \text{Absolute_Max_Time}) \vee (H_{\text{par_ema}} < \text{Absolute_Min_Ratio})$
- **Action:** Triggered strictly by failing chat health. The system recognizes the user is actively struggling to maintain the interaction. However, to prevent algorithmic dependence, the AI strictly avoids generating exact copy-paste scripts. Instead, it maximizes UI visibility to provide clear, structured, and tactical advice (e.g., framing techniques or specific topics to pivot to), acting as a conversational lifeline without doing the talking for them.

Case 3: MENTOR (The Weaning State)

- **Condition:** $S_{\text{rat_ema}} \geq 0.8$ or $\text{Final_S_inv} \geq 5$, advisor's condition is not met
- **Action:** Triggered *exclusively* by **High Dependency**. This is the "Synthetic Success" trap. Even if the chat latency and parity look perfect, the backend knows the human is offloading the cognitive work. The AI actively intervenes by degrading the generation UI and switching to Socratic mode (e.g., asking "What do you genuinely want to know about their last message?"). It forces the user to reclaim their interpersonal agency.
- **The Escape Hatch (Emergency Override):** If the user experiences severe cognitive overload despite the heuristic, they can trigger a secondary "Force Assist" option. This temporarily overrides the dependency lock, escalating the system to the **advisor** role for a single turn to provide concrete lifelines, before resetting the state. To prevent abuse, this option is placed behind intentional UI friction (e.g., a secondary confirmation dialogue).

Case 4: FACILITATOR (Organic Equilibrium & Success)

- **Condition:** $\text{Health} == \text{Stable/Optimal} \wedge \text{Dependency} == \text{Low}$
- **Action:** Triggered by **Organic Health**. The humans are doing well on their own. The AI stays out of the way. The UI minimizes to a highly unobtrusive state (e.g., a small

background icon). If clicked, it only offers light, thematic nudges rather than generating full sentences.

the precedence of roles are friend> Mentor>Advisor> facilitator

6. Frontend Visual Mapping

No dynamic UI should be necessary, a regular button and showing the suggestion at the top of the chat should be sufficient

7.database management

data are stored in the appwrite database, two things will be saved, the chat logs, as well as the semantic plan for the chat room,
chat log json format:

```
{
  "message_id": "chat_5512_9941_msg_88319", // the num in the back will be in
  sequential order
  "chat_id": "chat_5512_9941",
  "sender_id": "u_5512",
  "content": "Yeah, finals are brutal right now. You?",
  "char_count": 39,
  "is_ai_assisted": false,
  "created_at": "2026-04-04T11:30:15.000Z"
}
```

semantic plan json format:

```
{ "chat_id": "chat_5512_9941", //identifier, user_A id + user_B ID
  "current_role": "FRIEND", //the AI's role to dictate suggestion style
  "signal_state": {
    "h_lat_ema": 52.4,
    "h_par_ema": 0.88,
    "s_inv_penalties": 0 //keystroke
    "s_rat_ema": 0.15,
    "last_sinv_timestamp": "2026-04-03T20:00:00.000Z",
    "consecutive_healthy_messages": 4
  },
}
```

```

"context":{

"macro_summary": "Users just matched. User 1 mentioned being stressed about a final
project. User 2 hasn't responded yet.", //chat's overall context

"user_1_context": "", //profiling

"user_2_context": "", //profiling

}

"strategy":{

"strategic_intent": ""// the AI's reasoning and thinking

"action_plan": ""

}

"$updatedAt": "" //should just be native in appwrite

```

8.Context delay

Sinv = keystroke

Time based:

$$S_{inv} = S_{inv_new} = S_{inv} * (0.5)^{t/H}$$

H: time threshold where penalty is halved: default 3 hours

t: time elapsed

action based:

Sinv: For a certain number of consecutive message sent *without* triggering the Sinv penalty, subtract penalty: eg 10 messages sent by the user(**tracks a running total of the user's organic sends**) with out setting off the keystroke penalty, subtract 1 from the penalty count

problem:

context and compaction:

The solution could be structured extraction, a method which extracts key facts into a json structure and injects those facts.

how to pair with “Macro summary” field?:

macro summary is a summary done by other LLM, it may omit details and cause hallucination, so using key facts + summary may give the best of two worlds.

compaction strategy(agent 1):

The mechanism for triggering a summarization plus extraction is decoupled from the regular semantic plan update loop. working on separate triggers:

trigger A:

Raw message buffer of 30 messages, start compaction plus extraction if full, for cold start(first compaction) the activation is only 10 messages

trigger B:

time, if the current compaction is done over 3 hours ago, and the chat starts up again, activate the strategy to ensure it is up to date

trigger C:

State change, if the role changes activate the loop before hand to ensure complete context, but need some measure in place to prevent rapid activation due to role switching to quickly, something like a lock of 5 to 10 messages before Trigger C and be activated again

how to compact:

send the *current* Macro Summary, the *current* facts Structured JSON, and the *uncompressed buffer* of raw messages

perform a **Delta-Update**:

facts extraction:LLM use upsert to extract key facts and add new keys or update keys, keys are separated into immutable facts and ephemeral facts, immutable facts are upserted and not deleted, while ephemeral facts are Short-term context, overwritten or permanently deleted as soon as the conversation moves past them.

summary update: use the raw messages and the old macro summary, to rewrite the summary to include the new conversational arc.

Buffer cleaning:Once the Appwrite database successfully receives the updated Semantic Plan, clear the raw message buffer (except for perhaps the last 2-3 messages for immediate continuity).

What about the sliding window context for agent 2?:

is not plausible to give 29 messages to inject into a prompt as sliding window, so agent 2 uses a separate micro window(5-10 messages) instead, if the macro summary and facts have not been first updated, use the default
so we send the semantic plan, micron window, facts json to agent 2

json structure for facts:

"chat_id": "chat_5512_9941"

"immutable_facts": {"": ""},.....}

"ephemeral_facts":{"": ""},.....},

##Professor feedback: **FACTS json is unnecessary, just let the LLM do the macro context summary in the semantic plan should be enough, so maybe remove the 關鍵事實 JSON, and just keep everything else, relying on the macro summary and semantic plan**

Role for trouble makers:

in conjunction with the risk detection module, if a user is flagged as risky, a new role is assigned to them called "risky", in this role, suggestion becomes firm and let user know the consequences if they continue the bad action, it also becomes automatic instead of manual, telling user first: this behavior has been flagged, continue and you will be {consequences}, suggestion wise In this state both agents are disabled, refusing to help the user, if user click the help button, it should return a static, hardcoded string like: *"Assist disabled due to community guidelines violation. Please type your next message manually."*

transition:

In this role, transition will have nothing to do with signals, user will only transfer out of this role if :

they sent 5 messages that are clean, this should account for silence. If the user sends a toxic message, gets flagged, and then just closes the app out of frustration, or try to wait it out through time, the counter must not reset when the session expires. The required 5 clean messages must remain a persistent lock on their profile, forcing them to resolve the penalty whenever they log back in.

if the user successfully transfer out of the risky role, the default role is facilitator, and both agents turn back on, all the trigger counter from before the flagging should remain the same and continue as usual

if the user do not comply and continue malicious behavior:

Strike 1: Quarantine locked. Requires **5** consecutive clean messages to exit.

Strike 2: Quarantine locked. Requires **10** consecutive clean messages to exit (doubling the threshold is a standard, effective friction multiplier).

Strike 3: Terminal node. The chat room is instantly destroyed, and the user's account is flagged for a ban.

hardgate better than softgate:

see bounds section

NEW:

Inactive mode activates when a user idles for 5 minutes, prompting the AI to post a clearly branded system notification indicating the absence without progressing the conversational history, with the specific phrasing tailored to the AI's selected persona role. During this waiting period, a dual-sided drafting feature initiates to keep both participants supported. For the waiting user, a private workspace opens, transforming the dead time into a creative brainstorming session where the AI drafts tailored icebreakers or potential follow-up topics, dropping subtle hints toward fruitful avenues of connection without leaking background data. Simultaneously, the idle user's Copilot works in the background analyzing the recent chat history to proactively draft a contextual "welcome back" response, which is presented as a private suggestion the moment they return to their screen. If either user attempts to prompt the LLM to dig into sensitive profiling details, trauma, or unshared background data of their partner, the AI strictly declines the request and provides an explicit warning regarding

conversational boundaries. Once the idling user registers an active keystroke, the frontend immediately terminates the private drafting views, restores the standard input fields.

agent 1 message buffer trigger:

instead of a static 15 messages length buffer, which triggers agent 1 when full, the buffer should instead be dynamically sized, to do this we can evaluate the length based on volume and density:

for volume:

we define the max amount of token we would accept for the prompt to include all the messages alongside the mission statement and output limitation and other static things in the prompt, and we estimate the total token amount when we accept messages into the buffer, when it reach that parameter it triggers agent 1, replacing the buffer size trigger, but the buffer still need to exists to store the messages temporarily

for density(rubber banding)

adjust the trigger based on the agent 1's previous response, but first we would have to modify the agent 1's system prompt so it does not feel obligation to grab at least one triples, which ends up giving garbage, so we need to add a part about only grabbing info helpful to either identify/better understand the conversation topic, or the user's individual profiling, and that it is ok to return no triples, next we demand the agent give reasoning behind why it pic this triple, aka a significance score, something like:

```
{ "extracted_triples": [ { "subject": "user_1", "predicate": "IS_INTERESTED_IN", "object": "SKIING", "significance_score": 8, "reasoning": "Establishes a hobby for user_1 which improves profiling" } ] } .
```

after all of this the agent 1 returns a list of triples, which we filter through using our existent logic to check if the connection is in the allowed list, then we use that final number to rubber band the token threshold to either increase it if the chat is going slow, IE low or no new triples, or decrease it when it is fast IE high number of triples.

Remember to check if we have code to check if a triple already exists, if not make sure to implement it.

WARNING: for the triples filter, we may try to let similar connections like LIKE_TO and PREFERS_TO to be automatically converted to the ones in the allowed list, BUT since we will pass the allowed list to the LLM in prompt any ways, this seems redundant

8. Auditable AI & Provenance Lineage (Data Flow & Decoupling)

To guarantee system transparency and satisfy information-law traceability requirements, the architecture implements a formalized provenance framework. Because W3C PROV-DM JSON is extremely heavy, the actual generated suggestion (**nudge_text**) and the audit trail must be strictly decoupled by the backend before reaching the frontend.

Step 1: The Raw LLM Output (Agent 2)

When Agent 2 is invoked, it does not generate the W3C PROV-DM structure directly. Instead, the LLM is instructed via its system prompt to output a flat JSON containing the actual suggestion (conforming to its current role schema) alongside the specific reasoning variables it utilized.

Agent 2 Raw Output Schema:

JSON

```
{
  "nudge_type": "structural_lifeline",
  "nudge_text": "Validate her stress about the midterm, then pivot to asking how her MIS group is handling it.",
  "reasoning_variables": {
    "inferred_topic": "Academic Anxiety",
    "used_strategy_intent": "De-escalate tension by establishing mutual academic empathy.",
    "used_profile_fact": "MIS Undergraduate, graduation 2027.",
    "used_graph_edge": "User struggles with database architecture."
  }
}
```

Step 2: Backend Decoupling & Routing

The backend script intercepts Agent 2's raw output and splits the data into two completely separate pipelines to prevent UI latency and payload bloat.

Pipeline A: To Appwrite (audit_logs Collection) The backend extracts the `nudge_text` and `reasoning_variables`, combines them with system metadata (like `t_invoke_ms`, the exact LLM model, and the `current_role`), and constructs the rigid **W3C PROV-JSON** dictionary. This massive compliance payload (detailed in the W3C schema below) is sent directly to the database for permanent storage.

Pipeline B: To the Frontend Client (The UI DTO) The backend compiles the variables into a lightweight Data Transfer Object (DTO) for the React/Vue client. It uses a server-side string formatter to translate the `reasoning_variables` into a single human-readable paragraph.

Frontend API Response Schema:

JSON

```
{
  "suggestion_id": "sug_88192",
  "nudge_type": "structural_lifeline",
  "ui_nudge_text": "Validate her stress about the midterm, then pivot to asking how her MIS group is handling it.",
  "quick_trail_string": "Operating under the ADVISER framework, I used your MIS profile and the plan to 'de-escalate tension' to suggest a structural pivot regarding Academic Anxiety."
}
```

Step 3: Frontend Visual Mapping

When the frontend receives the DTO, it maps the two strings to separate UI components:

1. **The Primary Copilot Bubble:** Renders *only* the `ui_nudge_text`. This keeps the active chat clean and ensures the user's primary focus remains on the conversational advice.
2. **The Extendable Audit Bar:** A small `[+] View AI Reasoning` toggle is attached below the Copilot bubble. When clicked, an accordion drops down to display the `quick_trail_string`.
3. **The Data & Security Hub:** If the user wants the full, robust breakdown, they navigate to a separate tab in the app. This frontend page queries Appwrite for the full W3C PROV-JSON using the `suggestion_id`, completely isolating the heavy compliance data from the active chat interface.

TOKEN management: to make sure this does not cost extra token and increase latency we: You don't actually need Agent 2 to tell you what context it used, because **your backend already knows what context it gave to Agent 2**.

When a user clicks "Help me reply," your backend pulls the `semantic_plan` (macro summary & strategy) and the Knowledge Graph facts *before* making the API call. Because your backend is the one building the prompt, your backend can construct the audit trail deterministically without forcing the LLM to waste output tokens repeating facts back to you.

The Ultra-Lean Pipeline (Zero Extra Tokens)

Here is how you write this for your coding agent so it understands exactly how to build this without breaking your latency budget:

Step 1: Backend Gathers Variables (Pre-API Call)

The backend function initiates. It queries Appwrite and MongoDB, holding these variables in its local memory:

- `var_role` = "ADVISER"
- `var_strategy` = "De-escalate tension by establishing mutual academic empathy."
- `var_fact` = "MIS Undergraduate"

Step 2: The Lean Agent 2 API Call

The backend injects those variables into the system prompt and calls the LLM. Because of your strict Layer 2 Guardrails, the LLM outputs **only** what is necessary for the UI, keeping output tokens extremely low.

JSON

```
{
  "nudge_type": "structural_lifeline",
  "nudge_text": "Validate her stress about the midterm, then pivot to asking how her MIS group is handling it."
}
```

Step 3: Backend Assembles the Audit Trail (Post-API Call)

The LLM finishes in under 1.5 seconds. Now, the backend script takes over. It takes the LLM's `nudge_text` and marries it to the variables it already had in memory (`var_role`, `var_strategy`, `var_fact`).

The backend script (not the LLM) formats the `quick_trail_string`:

JavaScript

```
// Hardcoded backend logic, 0 token cost
const quick_trail_string = `Operating under the ${var_role} framework, I used your profile as
an ${var_fact} and the plan to '${var_strategy}' to formulate this structural pivot.`;
```

Step 4: Routing

Finally, the backend routes the data:

1. **To Frontend UI:** Sends the lean DTO { `"ui_nudge_text": "...",`
`"quick_trail_string": "..."` }.
2. **To Appwrite DB:** Injects `var_role`, `var_strategy`, `var_fact`, and the `nudge_text` into the hardcoded W3C PROV-JSON template and saves it to the `audit_logs` collection.

The Result: You get mathematically strict W3C compliance, a fully populated frontend audit UI, and **zero** extra output tokens added to Agent 2.

note:

Writing the massive W3C PROV-JSON to your Appwrite `audit_logs` collection *will* take a fraction of a second over the network (usually 50–150ms).

To prevent this from slowing down the UI, you must instruct your coding agent to make the database write **asynchronous (non-blocking)**. The backend must immediately return the DTO to the client *before* waiting for the Appwrite database to finish saving the audit log.